project-atlas command cheatsheet

a derived, auditable knowledgebase over your repository's documentation.

The markdown is the source of truth. Everything this tool writes is derived and safe to delete.

The shape of it

`init`	write project-atlas.config.json, detecting the layout and the plan document
`all`	scan + health + build
`serve`	build, then run the live dashboard detached and open it
`watch`	rebuild on change; --serve hosts it live at http://127.0.0.1:4173
`dashboard`	build, serve and hand back a URL

Lenses — read-only, and nothing they report is written into the site

git history	`contrib · ownership · surviving · git-insights · git-hotspots · git-history · git-branch · git-tree · git-status · git-diff`	
this working tree	`status · changes · diff · branch · review`	
the plan	`tasks · plan · design · contention · spec`	
your sessions	`sessions · tokens`	read from outside the repository, aggregate only, never published
continuity	`state · handoff · resume`	what a cold session reads first
agent surfaces	`mcp · ask · prompt`	
the tool itself	`help · version · config · caps`	

What writes, and what leaves this machine

writes in your repository	`init · build · note · worklog · branch · pause · stop · community`	everything build writes is derived and safe to delete
leaves this machine	`publish · artifact`	never without confirmation; nothing is pushed without --push
touches the network	`caps`	one request to the host API, and community makes the same one

Every other command on this card only reads.

Every command is `atlas <name>` in a shell and `/atlas:<name>` in Claude Code. ■ shell only □ Claude Code only

Aliases `capabilities = caps · git-insight = git-insights`

Flags, on every command

`--root <dir>`	repository root (default: git toplevel, else cwd)	`--no-git`	skip git metadata; staleness is reported as unchecked rather than guessed
`--config <path>`	config file (default: <root>/project-atlas.config.json)	`--offline`	skip the capability probe; assume features exist and say so
`--verbose[=all]`	list findings, not just counts	`--quiet`	suppress progress output
`--json`	machine-readable output	`--force`	(init) overwrite an existing config

every command, grouped by what you are trying to do

Every command is `atlas <name>` in a shell and `/atlas:<name>` in Claude Code. ■ shell only □ Claude Code only

Starting out

`build`	adopt this repository and generate the site — the one command that does a first run end to end
`help`	this list (also what a bare atlas prints)
`config [--json]`	the resolved configuration — the file merged over the defaults, which is what runs
`version`	which build is answering, where it lives, and whether it is behind

Seeing where things stand

□ `status`	the corpus, the rot and the one next action
`health [--verbose]`	report rot signals; exit 1 if any blocking signal fires
`tasks [word]`	the planning document with progress bars; a word filters it
`design [--scaffold]`	which design documents exist, and which are scaffolds still owing their substance
□ `dashboard`	build, serve and hand back a URL
`product`	every sibling repository under one directory at once — which have adopted this tool and which have not — for the case where the session is running from a directory that is not a repository, and every other surface here is therefore blank

Working on a change

`branch [type slug]`	branch state, or create type/short-slug carrying your changes
`plan [slug]`	the route for the change in your tree — branch, item, version; --apply cuts the branch
`changes [--json]`	what is uncommitted and which documents cite it
`diff <file>`	that file's diff — uncommitted, else across the branch
□ `review`	what this diff breaks or fixes, before the commit

Picking up cold

`state [--json]`	what a resuming session reads first: where you are, what was recorded
`handoff`	the derived half of a handoff, for the half only a person can write
`note <kind> "<text>"`	record one decision, trap or blocker — the only command here that writes to the repository

Digging into history

`contrib [--json]`	who did what, from git history alone
`ownership [--json]`	bus factor per area — how many people have ever touched it
`surviving [--json]`	whose written lines are still in the tree, by git blame rather than by commit
`worklog [--day D]`	write today's entry from git and the plan; --stdout to see it without writing
`sessions [--out F]`	how sessions went — turns, interruptions, friction, rework
`tokens [--out FILE]`	token accounting from local session transcripts — opt-in, never published
`--snapshot`	append the counts-only day rollup to .atlas/tokens.jsonl (needs tokens.snapshot)

What git history says, and nothing else here reads

`git-insights [sect]`	what git history says that nothing else here reads — strictly read-only sections: hotspots, coupling, branches, cadence, hygiene, change

What git history says, and nothing else here reads (cont.)

□ `git-hotspots`	the files change keeps returning to, what changes together, and who knows each area
□ `git-history`	cadence and commit hygiene
□ `git-branch`	branch state and the branch inventory
`git-tree`	the branch topology — what was cut from what, drawn as a tree, with every origin marked as the inference it is
□ `git-status`	where you are, what is uncommitted, and what this change touches
□ `git-diff <path>`	one file, with the history around it

Stopping and picking back up

`pause [--dry-run]`	checkpoint every agent worktree to a wip/agent-* ref before the session ends — writes git refs
`resume`	print the re-spawn plan for a paused session — branch, worktree, checkpoint
`stop [--force]`	clear session state and agent worktrees; every branch and checkpoint survives

Publishing

`caps`	which features the host has on — makes one network request
`community [--write]`	issue and PR scaffolding for the features that came back on
`publish`	wiki, pages or a single file, and never without confirmation
`--target wiki`	GitHub Wiki — flattened markdown, links rewritten, drift-guarded
`--target pages`	the full site to a gh-pages branch (dashboard + deck survive)
`--target export`	one self-contained HTML file (--page dashboard|index|health)
`--ci`	(GitLab, pages) write the .gitlab-ci.yml job instead — Pages is an artifact there
□ `artifact`	the dashboard, health report or index to claude.ai as a shareable page — it leaves this machine, with every local-only panel stripped and the export asserting that itself

Agent surfaces

`mcp`	serve the corpus over MCP on stdio — read-only, no dependency
`--status`	what a client would connect to, where it is registered, what is running
`ask <task>`	one structured answer for a program; exit 1 on findings, 2 if it could not run
`prompt [--out FILE]`	a system prompt assembled from this repository's own rules and state

Command line only

■ `init [--plan PATH]`	write project-atlas.config.json, detecting the layout and the plan document
■ `scan [--json]`	build and summarise the index
■ `contention [branch…]`	what a fan-out will collide on: files more than one branch touches, plan ids defined twice, and the next free id. --base REF; exit 1 on a duplicate id only
■ `spec --gate`	the commit gate: refuse a staged change whose message names no plan item
■ `serve`	build, then run the live dashboard detached and open it
`--status | --stop`	what is running here | end it now
`--list | --launcher`	every atlas dashboard on this machine | write a page linking them all
■ `watch [--serve]`	rebuild on change; --serve hosts it live at http://127.0.0.1:4173
■ `all`	scan + health + build